

1. A server stores event logs as characters. A log string contains lowercase letters only. The system compresses logs using this rule:

If 3 or more identical characters occur consecutively, they collapse into a single character.

This collapse may create a new sequence that also collapses.

The process continues until no more collapses are possible.

Example 1

Input:

aaabbbbc

Output:

abc

Example 2

Input:

abcccbbbaa

Now no group ≥ 3

Output:

acbaa

2. You are given n machines in a circle. Each machine has some load (integer).

In one move:

You may transfer 1 unit of load from machine i to machine $(i+1) \% n$. Each transfer costs 1. Your goal is to make all machines have equal load. Return the minimum cost, or -1 if impossible.

Example 1

Input:

[1, 2, 3]

Total = 6

Target = 2

Transfers:

Machine 3 sends 1 unit to Machine 1 (circular)

Cost = 1

Output:

1

Example 2

Input:

[4, 0, 0, 0]

Total = 4

Target = 1

Optimal transfers:

Machine 1 $\rightarrow$ Machine 2 (1)

Machine 1 $\rightarrow$ Machine 3 (1)

Machine 1 $\rightarrow$ Machine 4 (1)

Cost = 3

Output:

3

Example 3

Input:

[2, 2, 1]

Output:

-1

3. A string is called shift-valid if: Each character is either the same as previous OR exactly the next alphabet letter ('z' cannot shift forward) Return true if the string is shift-valid, else false.

Example 1

Input

s = "abbcd" (Note: the image shows 'abbcd' but the text says 'abbcd')

Output

true

Example 2

Input:

s = "abdz"

Output:

false

Example 3

Input:

s = "xyz"

Output:

true

4. You are given an integer n. Find the first digit (from left to right) that appears exactly once in the number. If no such digit exists return -1.

Input

n = 5722551

Output

7

Example 2

Input:

n = 998877

Output:

-1

5. You are given two 2D arrays representing the busy meeting intervals of two people during a day:

person1[][]

person2[][]

Each interval is [start, end] (times are integers, sorted or unsorted).

You are also given an integer duration representing the required meeting length.

Find the earliest common free time interval where both people are available for at least duration minutes.

If such a slot exists, return: start, start + duration]

Otherwise, return: -1

Input:

person1 = [[1,3], [6,7]]

```
person2 = [[2,4], [7,9]]  
duration = 1
```

Output:
[4,5]

Input:
person1 = [[1,5], [6,8]]
person2 = [[2,6], [8,10]]
duration = 2

Output:
-1